

COMM116

Generative AI Applications

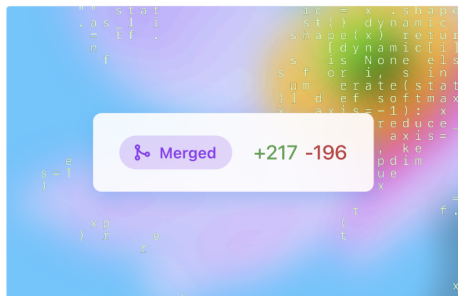
Extended Reading: AI Coding Agents

Dr Xiaoyang Wang

Department of Computer Science

x.wang7@exeter.ac.uk

How OpenAI uses Codex



Case Study: Codex at OpenAI



- **Internal Adoption:** Codex is utilized daily by Security, Product Engineering, Frontend, API, Infrastructure, and Performance Engineering teams in OpenAI.
- **Core Objectives:** Teams use the agent to manage complexity at scale, move faster, and improve work quality.
- **Primary Applications:**
 - Understanding complex systems and data flows.
 - Refactoring large codebases and updating legacy dependencies.
 - Resolving incidents fast.

Code Understanding & Navigation



- **Context Retrieval:** Codex replaces manual `grep` for “Where would I do this?” questions across diverse repos containing Terraform and Python.

Code Understanding & Navigation



- **Context Retrieval:** Codex replaces manual `grep` for “Where would I do this?” questions across diverse repos containing Terraform and Python.
- **Incident Response:**
 - Engineers paste **stack traces** directly into Codex to identify where specific logic, such as the **authentication flow**, resides.
 - It helps trace how failure states **propagate** across systems to accelerate triaging.

Code Understanding & Navigation

- **Context Retrieval:** Codex replaces manual `grep` for “Where would I do this?” questions across diverse repos containing Terraform and Python.
- **Incident Response:**
 - Engineers paste **stack traces** directly into Codex to identify where specific logic, such as the **authentication flow**, resides.
 - It helps trace how failure states **propagate** across systems to accelerate triaging.
- **System Mapping:** Agents can summarize request flows from endpoint to response and identify dependencies between service modules.

Refactoring & Migrations

Updating legacy codebases; making structural changes to a codebases; Code cleanup and modernization etc.

- **Large-scale Updates:** Used to apply changes consistently across dozens of files where simple regex or find-and-replace tools fail due to structural complexity.
- **Specific Transformations:**
 - Converting **callback-based database access** to modern **async/await** patterns (refactoring).
 - Swapping legacy modules for new service patterns and automatically opening Pull Requests (PRs).
- **Code Health:** Breaking up oversized modules and preparing legacy code for better testability (modularization).

- **Optimization:** Engineers prompt Codex to analyze slow or memory-intensive paths, identifying **redundant operations** or costly queries.
- **Database Efficiency:** Infrastructure teams scan for “hot paths” and **repeated expensive DB calls** to draft batched queries (resolve system bottlenecks).
- Draft batched versions of these repeated and expensive operations, which consolidate multiple individual requests into a single, more efficient operation.
- **Edge Case Testing:**
 - Generates unit/integration tests based on function signatures.
 - Specifically targets **boundary conditions** like empty inputs, max lengths, and null states.

Best Practices for Codex Workflows

- **Two-Step Flow (Ask Mode):** Start by prompting for an implementation plan in “Ask mode” before switching to “Code mode” for execution to keep the model grounded.
- **Persistent Context (AGENTS.md):** Maintaining an AGENTS.md file in the repository to provide Codex with naming conventions, business logic, and known quirks - link to the AI agents’ “Skills” idea.
- **Prompt Engineering:** Structure prompts to mirror **GitHub Issues**, including file paths, component names, and relevant diffs.
- **Best-of-N:** Simultaneously generating multiple responses for a single task to explore alternative solutions or combine the best elements of each.

Agent Skills?

According to the Cursor doc.

- **Definition:** Agent Skills is an open standard designed to extend AI agents with specialized, domain-specific knowledge and workflows.
- **Core Concept:** Skills act as portable, version-controlled packages that teach agents how to perform specific tasks using **scripts** and **templates**.
- **Key Characteristics:**
 - **Portable:** Works across any agent supporting the Agent Skills standard.
 - **Version-controlled:** Stored as files in repositories or installed via GitHub.
 - **Actionable:** Includes executable scripts and references.
 - **Progressive:** Loads resources on demand to keep context usage efficient.

How Skills Work

- **Discovery:** When Cursor starts, it automatically discovers skills from designated directories and makes them available to the Agent.
- **Relevance:** The agent evaluates the available skills and decides when to apply them based on the current context.
- **Invocation Methods:**
 - **Automatic calling:** The agent determines relevance autonomously.
 - **Manual:** Users can trigger skills by typing / in the Agent chat and searching for the skill name.

Skill Directories and Structure

- **Primary Locations:** Skills are loaded from Project-level (`.cursor/skills/`) or User-level (`/.cursor/skills/`) directories.
- **Compatibility:** Cursor also supports skills located in `.claude/skills/` or `.codex/skills/` directories.
- **File Structure:** Each skill must be a folder containing a **SKILL.md** file.
- **Optional Subdirectories:**
 - `scripts/`: Executable code in any language.
 - `references/`: Additional documentation loaded on demand.
 - `assets/`: Static resources like templates or images.

The SKILL.md



University
of Exeter

```
1 ---
2 name: my-skill
3 description: Short description of what this skill does and when to use it.
4 ---
5
6 # My Skill
7
8 Detailed instructions for the agent.
9
10 ## When to Use
11
12 - Use this skill when...
13 - This skill is helpful for...
14
15 ## Instructions
16
17 - Step-by-step guidance for the agent
18 - Domain-specific conventions
19 - Best practices and patterns
20 - Use the ask questions tool if you need to clarify requirements with the user
```

Advanced Integration: Scripts and Scripts

- **Execution:** The `scripts/` directory contains code the agent can run to perform tasks.
- **Language Support:** Scripts can be written in any format supported by the implementation, including Bash, Python, or JavaScript.
- **Self-Containment:** Scripts should handle edge cases gracefully and include helpful error messages.
- **Context Efficiency:** Keeping detailed material in separate files (references/assets) ensures the agent only loads information when necessary.

Managing Skills

- **Viewing Skills:** Discovered skills appear in Cursor Settings → Rules → Agent Decides.
- **GitHub Installation:** Users can import remote skills by adding a GitHub repository URL as a Remote Rule in settings.

Case Study: Claude Code at Anthropic

ANTHROPIC

How Anthropic teams use Claude Code



<https://www-cdn.anthropic.com/58284b19e702b49db9302d5b6f135ad8871e7658.pdf>

Transforming Workflows



- **Broad Adoption:** Used by developers and non-technical staff (Legal, Finance, Marketing) to automate complex projects and bridge skill gaps.
- **Impact:** Enabled 2-4x time savings in specialized domains and enabled staff to contribute to codebases outside their primary expertise.
- **Core Philosophy:** Moving from “one-off scripts” to persistent, agentic tools and autonomous development loops.

Data Infrastructure and Operations

- **Visual Debugging:** Teams feed screenshots of dashboards into Claude Code to diagnose Kubernetes issues.
- **Finance Automation:** Non-technical finance members use plain-text files to describe data workflows; the agent executes them to **produce Excel outputs**.
- **Onboarding:** New hires navigate massive codebases using Claude Code, which reads `Claude.md` files to explain data pipeline dependencies.
- **Parallelism:** Engineers run **multiple instances** across different repositories to maintain full context for various long-running projects.

Engineering Velocity and Product Development



- **Autonomous Loops:** Developers use “auto-accept mode” to let Claude write code, run builds, and iterate continuously.
- **RL Engineering:** Adopts a “try and rollback” methodology with frequent checkpoints, allowing the agent to attempt implementation and reverting if it diverges.
- **Legacy Support:** Handles repetitive tasks like removing “research preview” messages or updating character limits for marketing copy across codebases.

Security and Infrastructure Analysis



- **Incident Response:** Stack trace analysis reduced from 15 minutes of manual scanning to 5 minutes via Claude Code.
- **Terraform Reviews:** Security engineers ingest infrastructure plans and ask, "Am I going to regret this?" to identify security risks before approval.
- **Documentation Synthesis:** Ingests multiple sources to create markdown runbooks and troubleshooting guides for real-time debugging.
- **MCP Integration:** Uses Model Context Protocol (MCP) servers for sensitive data to maintain better security and logging control.

Bridging the Technical Skill Gap

- **Inference Team:** Reduced research time for complex ML concepts by 80% (from 1 hour of Google searching to 10-20 minutes).
- **Growth Marketing:** Built an agentic workflow that generates hundreds of Google Ads variations in minutes while adhering to strict character limits (30/90).
- **Product Design:** Designers directly implement “large state management changes” (manages complex modifications to the underlying logic that governs how data is stored, updated, and synchronized within a UI) and UI polish, bypassing the traditional design-to-eng handoff.

Strategic Best Practices

- **The “Slot Machine” Strategy:** Commit state, let the agent work autonomously for 30 minutes, then either accept or restart fresh.
- **Persistent Context:** Extensive use of `Claude.md` files to document tools, expectations, and prevent repeated mistakes (e.g., “run pytest not run”).
- **Specialized Sub-Agents:** Breaking complex workflows into separate agents (e.g., one for headlines, one for descriptions) for higher output quality.
- **Interactive Partnering:** Treating the agent as a collaborator rather than a “one-shot” solution, starting with minimal info and guiding the process - task breakdown, feedback loop, interactive improvement, prompt engineering.

New Trend: Long Running Autonomous Coding Agents



University
of Exeter

Beyond the Single Agent: Multi-Agent Systems

- **The Scaling Problem:** Long-running single agents eventually “lose the plot” as context fills and logic drifts.
- **The Solution:** Transition from a single agent to a **System of Agents**.
- **Specialized Sub-agents:**
 - Anthropic uses specialized sub-agents to handle granular tasks (e.g., separate agents for headlines and descriptions).
 - Cursor utilizes **Subagents** to manage parallel task execution and scoped context usage.
- **Efficiency:** Keeping agents narrowly scoped prevents the “drift” associated with massive, all-encompassing sessions.

Organizational Structure

Organizational Structure over Naive Swarms

- **The AI Org Chart:** Successful systems mirror organizations with clear roles: **Planner** → **Worker** → **Validator**.
- **Phase-Based Execution:**
 - **Ask Mode (Planner):** OpenAI teams start with a planning phase to ground the agent before execution.
 - **Code Mode (Worker):** The agent implements the established plan synchronously or autonomously.
- **Visibility:** Using structured plans and progress logs ensures ownership and keeps complex, multi-day projects moving.

Fresh Context & Test-Driven Control

- **Documentation as Memory:** Instead of relying on unreliable persistent memory, agents utilize stable artifacts:
 - **AGENTS.md / Claude.md:** Store naming conventions, business logic, and "quirks" to provide fresh context for every session.
 - **Cursor Skills:** Portable packages of domain knowledge loaded on demand.
- **Tests as the Steering Wheel:**
 - **Auto-Accept Mode:** Agents iterate through continuous loops of writing code and running tests.
 - **Validation Signals:** Using strong unit tests and regression checks as objective success metrics.

Reading

Cursor: Scaling long-running autonomous coding, <https://cursor.com/blog/scaling-agents>

Building a C compiler with a team of parallel Claudes,
<https://www.anthropic.com/engineering/building-c-compiler>

Factory: Missions, <https://docs.factory.ai/welcome>